Adaptive Drone Swarm Behavior Simulation: Project Report

Anup Baigain, Madhusudan Pun, Toran Bahadur Niroula

Date: May 8, 2025

Abstract
This project presents the development of a 3D drone swarm simulation using Python, designed to replicate emergent behaviors inspired by natural swarm intelligence. The simulation incorporates key behavioral algorithms, including alignment, cohesion, separation, obstacle avoidance, and dynamic pattern formation, allowing drones to navigate complex environments collaboratively. Implemented using object-oriented programming and visualized with `matplotlib` in a 3D space, the system enables intuitive observation of swarm dynamics. The modular design supports expansion into more advanced features such as reinforcement learning and role-based tasking. This work serves as both a demonstration of core Python programming skills acquired through the Samsung Innovation Campus and a foundation for future research in multi-agent systems and autonomous aerial robotics.

1 Introduction

This project delivers a modular simulation environment in Python to model and visualize adaptive drone swarm behaviors inspired by biological systems. The simulation demonstrates emergent behaviors such as cohesion, alignment, separation, obstacle avoidance, and dynamic pattern formation, aiming to bridge the gap between theoretical swarm algorithms and practical implementation in complex environments.

2 Project Objectives and Scope

Core Objectives:

- Develop a modular simulation platform for drone swarms.
- Implement fundamental swarm behaviors: cohesion, alignment, and separation.
- Design adaptive collision avoidance.
- Enable dynamic pattern formation (circle, grid, triangle, etc.).
- Extend to 3D simulation and explore reinforcement learning for adaptability (as time allows).

Scope & Deliverables:

- Physics-based drone movement and collision detection.
- Swarm intelligence algorithms (cohesion, alignment, separation).
- Obstacle avoidance for both static (pillars) and dynamic (other drones) obstacles.
- User-selectable pattern formations.

- Visualization and interactive GUI.
- Technical documentation and performance analysis.

3 Software Architecture Overview

The system is organized into several key modules:

- **Physics Engine:** Simulates drone kinematics, velocity, and collision detection.
- **Swarm Behavior Module:** Implements Boids-inspired rules for cohesion, alignment, and separation, as well as PID-based goal seeking and obstacle avoidance.
- **Visualization System:** Uses Tkinter and Matplotlib for real-time 3D visualization and interactive controls.
- **Scenario Manager:** Configures start/end zones, obstacles (pillars), and pattern targets.

4 Implementation Details

4.1 Core Classes

- **Pillar:** Represents cylindrical obstacles in the simulation space, supporting collision checks.
- **Drone:** Models individual drones with position, velocity, and behavior logic, including:
  - Pattern Formation: Uses PD control to move drones into geometric patterns (circle, line, square, star, triangle, etc.).
  - Obstacle Avoidance: Combines PID control for goal seeking with Boids rules for inter-drone avoidance and obstacle repulsion.
- **Swarm:** Manages the collection of drones, scenario setup, pattern assignment, and simulation updates.
- **SimulatorGUI:** Provides an interactive GUI for mode switching (pattern formation/obstacle avoidance), parameter tuning, and real-time visualization.

4.2 Swarm Behaviors

- **Cohesion/Alignment/Separation:**
  In the drone swarm simulation, three fundamental behaviors govern the collective movement of the drones. Cohesion ensures that each drone steers toward the average position of its neighboring drones, maintaining swarm integrity and preventing fragmentation. Alignment causes drones to match velocities with nearby swarm members, resulting in synchronized and coordinated group movement. Separation maintains a minimum safe distance between drones through repulsion forces, effectively preventing collisions while allowing the swarm to navigate complex environments.

```
def calculate_avoidance_force(self, swarm, obstacles):
    avoidance_force = np.array([0.0, 0.0, 0.0])
    swarm_center = np.zeros(3)
    neighbor_count = 0

    # Boids implementation
    for other_drone in swarm.drones:
        dist = np.linalg.norm(self.position - other_drone.position)
        if dist < self.avoidance_radius:
            swarm_center += other_drone.position
            neighbor_count += 1
            avoidance_force += (self.position - other_drone.position)/dist**2
     # Separation

    if neighbor_count > 0:
        cohesion_force = (swarm_center/neighbor_count - self.position) * 0.1
    # Cohesion
         alignment_force = (np.mean([d.velocity for d in neighbors], axis=0) -
     self.velocity) * 0.1  # Alignment
```

- **Pattern Formation:** Drones can arrange themselves into user-selected formations, with smooth transitions between patterns. It uses matrix operations similar to Swarm-in-Blocks' formation system, For example:

```
def generate_pattern_targets(self, pattern_name):
    if pattern_name == "circle":
        radius = 7.5
        return np.array([[radius*np.cos(), radius*np.sin(), 0]
                         for  in np.linspace(0, 2, num_drones)])
    elif pattern_name == "square":
        return np.array([[x, y, 0] for x in [-5,0,5] for y in [-5,0,5]])
```

- **Obstacle Avoidance:** Drones detect and avoid pillars and other drones using repulsion forces and PID control.

```
# Pillar collision detection
dist_to_axis_sq = dx**2 + dy**2
is_within_height = position[2] <= self.height
return dist_to_axis_sq <= radius**2 and is_within_height

# Drone.update()
if obstacle.is_inside(drone.position):
    drone.eliminated = True
```

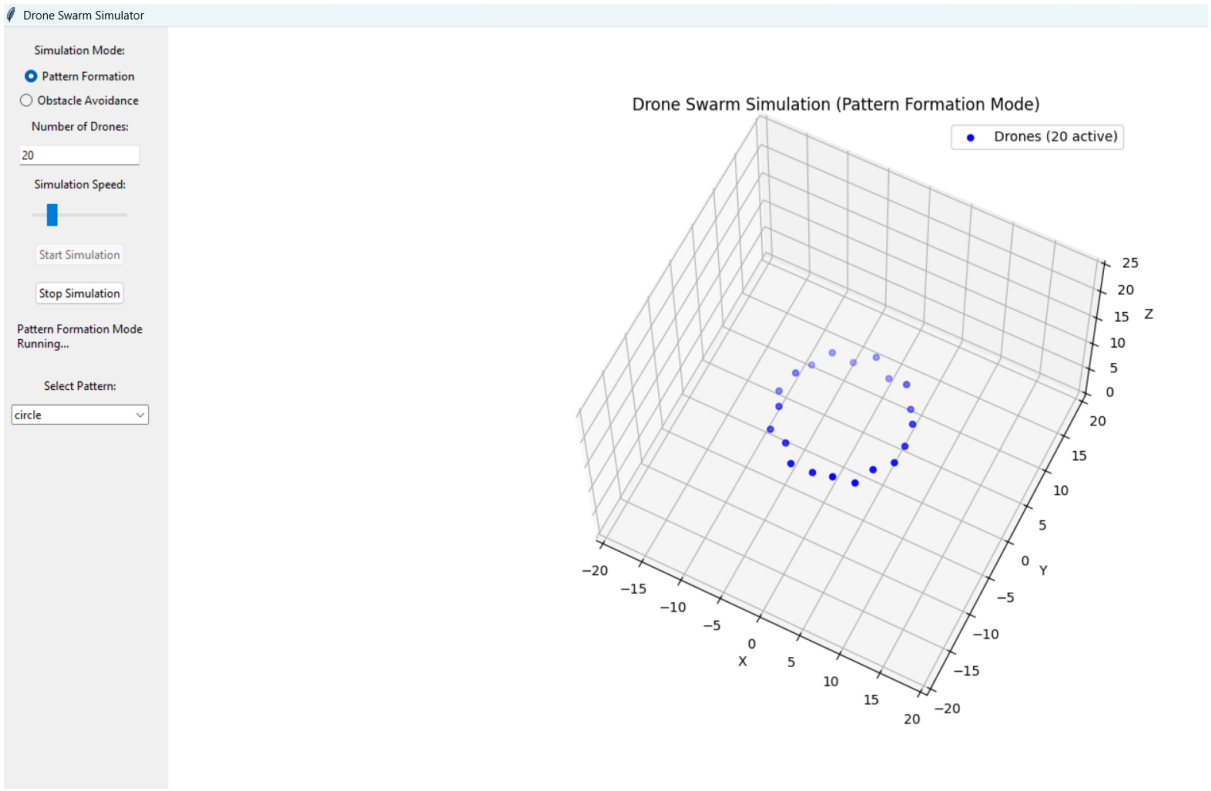


Figure 1: Circular pattern in 3D space

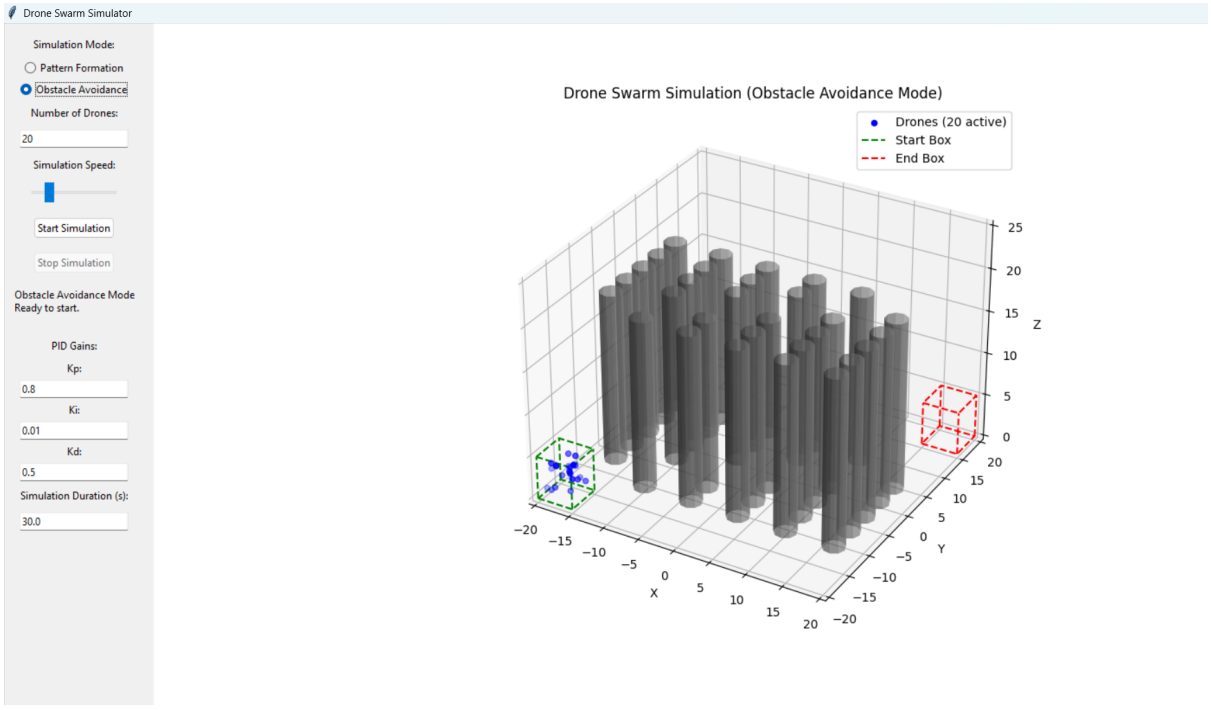


Figure 2: Pillar as obstacle in 3D space

4.3 Advanced Features

- **Hybrid Control System:**
  It combines PD control for pattern tracking with PID for goal seeking. It implements mode switching through GUI.

```
# Drone.update()
if mode == 'pattern_formation':
    force += calculate_pattern_force()
elif mode == 'obstacle_avoidance':
    force += calculate_pid_force() + calculate_avoidance_force()
```

- **Adaptive PID Controller:**
  Tunable gains (Kp=0.8, Ki=0.01, Kd=0.5) enable dynamic response adjustment. Includes anti-windup through velocity checks.

```
# PID force calculation
P_term = Kp * error
I_term = Ki * integral_error
D_term = Kd * derivative_error
```

- **3D Visualization:**
  Uses Matplotlib 3D projection for real-time visualization. Also, Renders obstacles as translucent cylinders.

```
# GUI.plot_swarm()
self.ax_swarm = self.figure.add_subplot(111, projection='3d')
ax.scatter(x,y,z, marker='o', c='b')
```

4.4 User Interaction

- Switch between ”Pattern Formation” and ”Obstacle Avoidance” modes.
- Select pattern type and number of drones.
- Adjust PID controller gains and simulation speed.
- Start, stop, and reset simulations with real-time feedback and results display.

5 Results and Features

- **Emergent Swarm Behavior:** The simulation demonstrates realistic flocking, pattern formation, and adaptive navigation through obstacle fields.
- **3D Visualization:** Real-time 3D plotting of drones, obstacles, and start/end zones.
- **Performance Metrics:** Tracks success rates, elimination counts, and average times for drones reaching goals.
- **Extensibility:** Modular codebase supports future integration of advanced features like leader election, adaptive formation switching, and reinforcement learning.

6 Conclusion

The project successfully meets its core objectives by providing a robust, interactive, and extensible platform for simulating and analyzing drone swarm behaviors. The implementation closely follows the proposed architecture and fulfills the planned deliverables, serving as a valuable tool for research and demonstration of multi-agent systems and swarm intelligence.

References

[1] swarm lab

[2] tkinter documentation

[3] matplotlib documentation

[4] SwarmLab: a MATLAB Drone Swarm Simulator

A Appendix

```
import tkinter as tk
from tkinter import ttk
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import time
import threading
import math
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg


class Pillar:
    """Represents a cylindrical pillar obstacle in the simulation."""
    def __init__(self, base_position, height, radius):
        self.base_position = np.array(base_position, dtype=float)
        self.height = height
        self.radius = radius

    def is_inside(self, position):
        """Checks if a given 3D position is inside the pillar."""
        dx = position[0] - self.base_position[0]
        dy = position[1] - self.base_position[1]
        dist_sq = dx**2 + dy**2
        is_within_radius = dist_sq <= self.radius**2
        is_within_height = (position[2] >= self.base_position[2] and
                            position[2] <= self.base_position[2] + self.height)
        return is_within_radius and is_within_height


class Drone:
    """Represents an individual drone in the swarm."""
    def __init__(self, drone_id, position, velocity=None):
        self.id = drone_id
        self.position = np.array(position, dtype=float)
        self.velocity = np.array(velocity if velocity is not None else [0.0, 0.0,
    0.0], dtype=float)
        self.target_position = np.array(position, dtype=float)
        self.max_speed = 4.0
        self.arrival_threshold_pattern = 0.2
        self.Kp_pattern = 0.5
        self.Kd_pattern = 0.5
        self.goal_seeking_strength_boids = 0.4
        self.cohesion_strength = 0.1
        self.alignment_strength = 0.1
        self.avoidance_radius = 3.0
        self.repulsion_strength = 2.5
        self.Kp_pid = 0.8
        self.Ki_pid = 0.01
        self.Kd_pid = 0.5
        self.integral_error = np.array([0.0, 0.0, 0.0])
        self.arrival_threshold_goal = 2.0
        self.eliminated = False
        self.reached_goal = False
        self.time_to_goal = -1.0
        self.path_taken = []
        self.broadcast_info = {}
        self.previous_error = np.array([0.0, 0.0, 0.0])

    def calculate_pattern_force(self):
        """Calculates the force to move the drone towards its target_position in
    pattern formation mode."""
```

```
        position_error = self.target_position - self.position
        velocity_error = np.array([0.0, 0.0, 0.0]) - self.velocity
        pattern_force = self.Kp_pattern * position_error + self.Kd_pattern *
velocity_error
        distance_to_target = np.linalg.norm(position_error)
        if distance_to_target < self.arrival_threshold_pattern and np.linalg.norm(
self.velocity) < 0.1:
            return np.array([0.0, 0.0, 0.0])

        force_magnitude = np.linalg.norm(pattern_force)
        if force_magnitude > self.max_speed:
            pattern_force = pattern_force / force_magnitude * self.max_speed
        return pattern_force

    def calculate_pid_force(self, goal_position, dt):
        """Calculates force using a PID controller towards the goal_position."""
        error = goal_position - self.position
        P_term = self.Kp_pid * error
        self.integral_error += error * dt
        I_term = self.Ki_pid * self.integral_error
        derivative_error = (error - self.previous_error) / dt if dt > 0 else np.
array([0.0, 0.0, 0.0])
        D_term = self.Kd_pid * derivative_error
        self.previous_error = error.copy()
        pid_force = P_term + I_term + D_term
        distance_to_goal = np.linalg.norm(error)
        if distance_to_goal < self.arrival_threshold_goal and np.linalg.norm(self.
velocity) < 0.1:
            return np.array([0.0, 0.0, 0.0])

        force_magnitude = np.linalg.norm(pid_force)
        if force_magnitude > self.max_speed:
            pid_force = pid_force / force_magnitude * self.max_speed
        return pid_force

    def calculate_goal_force_boids(self, goal_position):
        """Calculates the force towards a fixed goal_position for Boids-like
behavior."""
        direction_to_goal = goal_position - self.position
        distance_to_goal = np.linalg.norm(direction_to_goal)

        if distance_to_goal > self.arrival_threshold_goal:
            goal_force = direction_to_goal / distance_to_goal * self.
goal_seeking_strength_boids
            return goal_force
        else:
            return np.array([0.0, 0.0, 0.0])

    def calculate_avoidance_force(self, swarm, obstacles):
        """Calculates forces for avoiding other drones and obstacles."""
        avoidance_force = np.array([0.0, 0.0, 0.0])
        neighbor_positions = []
        neighbor_velocities = []
        swarm_center = np.array([0.0, 0.0, 0.0])
        neighbor_count = 0

        for other_drone in swarm.drones:
            if other_drone.id != self.id and not other_drone.eliminated and not
other_drone.reached_goal:
                dist_between_drones = np.linalg.norm(self.position - other_drone.
position)
                if dist_between_drones < self.avoidance_radius:
                    neighbor_positions.append(other_drone.position)
```

```
                neighbor_velocities.append(other_drone.velocity)
                swarm_center += other_drone.position
                neighbor_count += 1
                if dist_between_drones > 1e-6:
                    repel_direction = (self.position - other_drone.position) /
dist_between_drones
                    avoidance_force += repel_direction * (self.
repulsion_strength / dist_between_drones**2)

    cohesion_force = np.array([0.0, 0.0, 0.0])
    alignment_force = np.array([0.0, 0.0, 0.0])

    if neighbor_count > 0:
        average_position = swarm_center / neighbor_count
        cohesion_force = (average_position - self.position) * self.
cohesion_strength

        average_velocity = np.mean(neighbor_velocities, axis=0)
        if np.linalg.norm(average_velocity) > 1e-6:
            alignment_force = (average_velocity / np.linalg.norm(
average_velocity) -
                               self.velocity / np.linalg.norm(self.velocity + 1
e-6)) * self.alignment_strength

    for obstacle in obstacles:
        dx = self.position[0] - obstacle.base_position[0]
        dy = self.position[1] - obstacle.base_position[1]
        dist_to_axis_sq = dx**2 + dy**2
        dist_to_axis = np.sqrt(dist_to_axis_sq)

        is_within_pillar_height = (self.position[2] >= obstacle.base_position
[2] and
                                   self.position[2] <= obstacle.base_position
[2] + obstacle.height)

        if dist_to_axis < (obstacle.radius + self.avoidance_radius) and
is_within_pillar_height:
            if dist_to_axis > 1e-6:
                repel_direction_xy = np.array([dx, dy, 0]) / dist_to_axis
                avoidance_strength = self.repulsion_strength * (1.0 / (
dist_to_axis - obstacle.radius + 1e-6))
                avoidance_force += repel_direction_xy * avoidance_strength

    return avoidance_force, cohesion_force, alignment_force

def update(self, dt, swarm):
    """Updates the drone's state (position, velocity) based on calculated
forces."""
    if self.eliminated or self.reached_goal:
        return

    total_force = np.array([0.0, 0.0, 0.0])

    if swarm.mode == 'pattern_formation':
        total_force += self.calculate_pattern_force()
        avoidance_force, _, _ = self.calculate_avoidance_force(swarm, [])
        total_force += avoidance_force
    elif swarm.mode == 'obstacle_avoidance':
        total_force += self.calculate_pid_force(swarm.end_box_center, dt)
        avoidance_force, _, _ = self.calculate_avoidance_force(swarm, swarm.
obstacles)
        total_force += avoidance_force
        self.path_taken.append(self.position.copy())
```

```
        self.velocity += total_force * dt
        speed = np.linalg.norm(self.velocity)
        if speed > self.max_speed:
            self.velocity = self.velocity / speed * self.max_speed
        self.position += self.velocity * dt
        if swarm.mode == 'obstacle_avoidance':
            if (self.position[1] < swarm.start_box_center[1] - swarm.start_box_size
    / 2 - 2 or
                    self.position[2] < -2 or self.position[2] > swarm.area_size / 2
    + 2):
                self.eliminated = True
                print(f"Drone␣{self.id}␣eliminated␣due␣to␣going␣out␣of␣bounds.")

            end_box_min = swarm.end_box_center - swarm.end_box_size / 2.0
            end_box_max = swarm.end_box_center + swarm.end_box_size / 2.0
            if np.all(self.position >= end_box_min) and np.all(self.position <=
    end_box_max):
                self.reached_goal = True
                self.velocity = np.array([0.0, 0.0, 0.0])
                self.time_to_goal = swarm.elapsed_simulation_time
                print(f"Drone␣{self.id}␣reached␣the␣goal␣at␣time␣{self.time_to_goal
    :.2f}␣s.")
                path_len = 0
                if len(self.path_taken) > 1:
                    for i in range(len(self.path_taken) -1):
                        path_len += np.linalg.norm(self.path_taken[i+1] - self.
    path_taken[i])
                self.broadcast_info = {'id': self.id, 'cost': self.time_to_goal, '
    path_length': path_len}


class Swarm:
    """Manages a collection of drones and the overall simulation state."""
    def __init__(self, num_drones, start_box_size=5, end_box_size=5, area_size=40):
        self.num_drones = num_drones
        self.start_box_size = start_box_size
        self.end_box_size = end_box_size
        self.area_size = area_size

        self.start_box_center = np.array([-area_size / 2 + start_box_size / 2,
                                          -area_size / 2 + start_box_size / 2,
                                          start_box_size / 2], dtype=float)
        self.end_box_center = np.array([area_size / 2 - end_box_size / 2,
                                        area_size / 2 - end_box_size / 2,
                                        end_box_size / 2], dtype=float)

        self.mode = 'pattern_formation'
        self.obstacles = []
        self.drones = []
        self.current_pattern_targets = []

        self.transitioning = False
        self.transition_start_time = 0
        self.transition_duration = 3.0
        self.start_transition_targets = []
        self.end_transition_targets = []

        self.simulation_speed = 1.0
        self.simulation_duration = 30.0
        self.elapsed_simulation_time = 0.0
        self.gui = None
        self.generate_obstacles()
```

```
    def get_random_start_position(self):
        """Generates a random position within the start box."""
        x = np.random.uniform(self.start_box_center[0] - self.start_box_size / 2,
                              self.start_box_center[0] + self.start_box_size / 2)
        y = np.random.uniform(self.start_box_center[1] - self.start_box_size / 2,
                              self.start_box_center[1] + self.start_box_size / 2)
        z = np.random.uniform(self.start_box_center[2] - self.start_box_size / 2,
                              self.start_box_center[2] + self.start_box_size / 2)
        return np.array([x, y, z])

    def generate_obstacles(self):
        """Generates obstacles (pillars) for the obstacle avoidance mode."""
        self.obstacles = []
        if self.mode == 'obstacle_avoidance':
            num_rows = 6
            num_cols = 5
            pillar_height = self.area_size / 2
            pillar_radius = 1.5
            row_spacing = (self.area_size - self.start_box_size - self.end_box_size
    ) / (num_rows + 2)
            col_spacing = self.area_size / (num_cols + 1)
            y_start = self.start_box_center[1] + self.start_box_size / 2 +
    row_spacing

            for r in range(num_rows):
                y = y_start + r * row_spacing
                gap_col = r % (num_cols + 1)
                for c in range(num_cols + 1):
                    x = -self.area_size / 2 + c * col_spacing + col_spacing / 2
                    if c != gap_col:
                        self.obstacles.append(Pillar([x, y, 0], pillar_height,
    pillar_radius))

    def generate_pattern_targets(self, pattern_name, size=15):
        """Generates target positions for drones to form a specified pattern."""
        targets = []
        active_drones = [d for d in self.drones if not d.eliminated]
        n_active = len(active_drones)
        if n_active == 0:
            return []
        center_point = np.array([0, 0, self.area_size / 4])
        if pattern_name == "line":
            line_length = size
            targets = np.array([[i, 0, 0] for i in np.linspace(-line_length / 2,
    line_length / 2, n_active)])
        elif pattern_name == "circle":
            radius = size / 2.0
            angle_step = 2 * np.pi / n_active if n_active > 0 else 0
            targets = np.array([[radius * np.cos(i * angle_step),
                                 radius * np.sin(i * angle_step), 0] for i in range
    (n_active)])
        elif pattern_name == "square":
            side_length = int(np.ceil(np.sqrt(n_active)))
            spacing = size / (side_length - 1 if side_length > 1 else 1) if
    side_length > 0 else 0
            center_offset = (side_length - 1) * spacing / 2
            targets = np.array([[(i % side_length) * spacing - center_offset,
                                 (i // side_length) * spacing - center_offset, 0]
    for i in range(n_active)])
        elif pattern_name == "star":
            radius = size / 2.0
            inner_radius = radius / 2.5
            points_per_star = 5
```

```
            temp_targets_list = []
            for i in range(n_active):
                angle_outer = 2 * np.pi * (i % points_per_star) / points_per_star
                angle_inner = angle_outer + np.pi / points_per_star
                if i % 2 == 0:
                    x = radius * np.cos(angle_outer)
                    y = radius * np.sin(angle_outer)
                else:
                    x = inner_radius * np.cos(angle_inner)
                    y = inner_radius * np.sin(angle_inner)
                temp_targets_list.append(np.array([x, y, 0]))
            targets = np.array(temp_targets_list)
        elif pattern_name == "triangle":
            side_length = size
            height = side_length * np.sqrt(3) / 2
            drones_per_side = n_active // 3 if n_active >= 3 else n_active
            if drones_per_side == 0: drones_per_side = 1
            count = 0
            temp_targets_list = []

            for i in range(drones_per_side):
                x_val = -side_length / 2 + i * (side_length / (drones_per_side - 1
    if drones_per_side > 1 else 1))
                if count < n_active: temp_targets_list.append(np.array([x_val, -
    height / 2, 0])); count += 1
            for i in range(drones_per_side):
                x_val = -side_length / 2 + i * ( (0 - (-side_length/2)) / (
    drones_per_side - 1 if drones_per_side > 1 else 1))
                y_val = -height / 2 + i * ( (height/2 - (-height/2)) / (
    drones_per_side - 1 if drones_per_side > 1 else 1))
                if count < n_active: temp_targets_list.append(np.array([x_val,
    y_val, 0])); count += 1
            for i in range(drones_per_side):
                x_val = side_length / 2 + i * ( (0 - (side_length/2)) / (
    drones_per_side - 1 if drones_per_side > 1 else 1))
                y_val = -height / 2 + i * ( (height/2 - (-height/2)) / (
    drones_per_side - 1 if drones_per_side > 1 else 1))
                if count < n_active: temp_targets_list.append(np.array([x_val,
    y_val, 0])); count += 1
            while count < n_active:
                temp_targets_list.append(np.array([0,0,0])); count +=1
            targets = np.array(temp_targets_list)

        elif pattern_name == "1":
            height = size
            targets = np.array([[0, -height / 2 + i * (height / (n_active - 1 if
    n_active > 1 else 1)), 0]
                                for i in range(n_active)])
        elif pattern_name == "S":
            length = size
            segments = 3
            points_per_segment = n_active // segments if n_active >= segments else
    n_active
            if points_per_segment == 0: points_per_segment = 1
            count = 0
            y_start = -length / 2
            segment_length = length / segments
            temp_targets_list = []
            for s in range(segments):
                x_offset = (s % 2) * length / 4
                y_segment_start = y_start + s * segment_length
                for i in range(points_per_segment):
                    if count < n_active:
```

```
                    x = x_offset + (-length/8 if s % 2 == 0 else length/8) * np
.sin(i * np.pi / (points_per_segment - 1 if points_per_segment > 1 else 1))
                    y = y_segment_start + i * (segment_length / (
points_per_segment - 1 if points_per_segment > 1 else 1))
                    temp_targets_list.append(np.array([x,y,0]))
                    count += 1
            while count < n_active:
                temp_targets_list.append(np.array([0,0,0])); count += 1
            targets = np.array(temp_targets_list)
        else:
            targets = np.random.rand(n_active, 3) * size - size / 2
            targets[:, 2] = np.random.rand(n_active) * size
        targets += center_point
        final_targets = []
        target_index = 0
        for drone in self.drones:
            if not drone.eliminated:
                if target_index < len(targets):
                    final_targets.append(targets[target_index])
                    target_index += 1
                else:
                    final_targets.append(drone.position.copy())
        return final_targets

    def set_pattern(self, pattern_name, size=15):
        """Sets a new pattern for the drones to form, initiating a transition."""
        if self.mode != 'pattern_formation':
            print("Pattern␣setting␣is␣only␣available␣in␣Pattern␣Formation␣mode.")
            return
        active_drones = [d for d in self.drones if not d.eliminated]
        if not active_drones:
            print("No␣active␣drones␣to␣set␣pattern␣for.")
            return
        new_targets = self.generate_pattern_targets(pattern_name, size)

        while len(new_targets) < len(active_drones):
            new_targets.append(active_drones[-1].position.copy())
        new_targets = new_targets[:len(active_drones)]
        self.start_transition_targets = [drone.target_position.copy() for drone in
active_drones]
        self.end_transition_targets = new_targets
        self.transition_start_time = time.time()
        self.transitioning = True

    def update_swarm(self, dt):
        """Updates the state of all drones in the swarm."""
        if not self.gui or not self.gui.running:
            return

        current_time = time.time()
        active_drones = [d for d in self.drones if not d.eliminated and not d.
reached_goal]
        if self.mode == 'pattern_formation':
            if self.transitioning:
                elapsed_time = current_time - self.transition_start_time
                transition_progress = min(elapsed_time / self.transition_duration,
1.0)
                for i, drone in enumerate(active_drones):
                    if i < len(self.start_transition_targets) and i < len(self.
end_transition_targets):
                        drone.target_position = (self.start_transition_targets[i] +
                                                 (self.end_transition_targets[i] -
self.start_transition_targets[i]) * transition_progress)
```

```
            if transition_progress >= 1.0:
                self.transitioning = False
                target_idx = 0
                for drone_idx, drone_obj in enumerate(self.drones):
                    if not drone_obj.eliminated:
                        if target_idx < len(self.end_transition_targets):
                            while drone_idx >= len(self.current_pattern_targets
):
                                self.current_pattern_targets.append(np.array
([0.0,0.0,0.0]))
                            self.current_pattern_targets[drone_idx] = self.
end_transition_targets[target_idx].copy()
                            target_idx +=1
        else:
            target_idx = 0
            for drone_idx, drone_obj in enumerate(self.drones):
                if not drone_obj.eliminated:
                    if target_idx < len(self.current_pattern_targets):
                        while drone_idx >= len(self.current_pattern_targets):
                            self.current_pattern_targets.append(np.array
([0.0,0.0,0.0]))
                        drone_obj.target_position = self.
current_pattern_targets[drone_idx].copy()


    for drone in active_drones:
        drone.update(dt, self)

    if self.mode == 'obstacle_avoidance':
        self.elapsed_simulation_time += dt
        drones_to_keep = []
        eliminated_count = 0
        reached_count = 0
        for drone in self.drones:
            if not drone.eliminated and not drone.reached_goal:
                collided = False
                for obstacle in self.obstacles:
                    if obstacle.is_inside(drone.position):
                        drone.eliminated = True
                        collided = True
                        eliminated_count += 1
                        break
                if not collided:
                    drones_to_keep.append(drone)
            elif drone.reached_goal:
                drones_to_keep.append(drone)
                reached_count += 1
        self.drones = drones_to_keep
        if self.elapsed_simulation_time >= self.simulation_duration or \
           (len(self.drones) == reached_count + eliminated_count and self.
num_drones > 0 and len(active_drones) == 0):
            if self.gui:
                self.gui.end_simulation_run()

def resize_swarm(self, new_num_drones):
    """Resizes the swarm to a new number of drones."""
    self.num_drones = new_num_drones
    self.drones = [Drone(i, self.get_random_start_position()) for i in range(
self.num_drones)]

    if self.mode == 'pattern_formation':
        if hasattr(self, 'gui') and self.gui and self.gui.pattern_var.get():
```

```
            self.current_pattern_targets = self.generate_pattern_targets(self.
gui.pattern_var.get())
        else:
            self.current_pattern_targets = [drone.position.copy() for drone in
self.drones]
    else:
        self.current_pattern_targets = [drone.position.copy() for drone in self
.drones]
    self.transitioning = False

def set_mode(self, mode):
    """Sets the simulation mode (pattern_formation or obstacle_avoidance)."""
    if mode not in ['pattern_formation', 'obstacle_avoidance']:
        print(f"Invalid mode: {mode}")
        return

    self.mode = mode
    self.generate_obstacles()
    self.reset_swarm()
    print(f"Switched to {mode} mode.")
    if self.gui:
        if self.mode == 'pattern_formation':
            self.gui.info_label.config(text="Pattern Formation Mode")
        else:
            self.gui.info_label.config(text="Obstacle Avoidance Mode\nReady to
start.")

def reset_swarm(self):
    """Resets all drones to their initial states and positions."""
    self.drones = [Drone(i, self.get_random_start_position()) for i in range(
self.num_drones)]
    if self.mode == 'pattern_formation':
        if hasattr(self, 'gui') and self.gui and self.gui.pattern_var.get():
            self.current_pattern_targets = self.generate_pattern_targets(self.
gui.pattern_var.get())
        else:
            self.current_pattern_targets = [drone.position.copy() for drone in
 self.drones]
    else:
        self.current_pattern_targets = [drone.position.copy() for drone in self
.drones]
    self.transitioning = False
    self.elapsed_simulation_time = 0.0

def calculate_run_results(self):
    """Calculates and prints the results of an obstacle avoidance simulation
run."""
    successful_drones = [d for d in self.drones if d.reached_goal]
    num_successful = len(successful_drones)
    total_initial_drones = self.num_drones


    num_eliminated = total_initial_drones - num_successful - len([d for d in
self.drones if not d.reached_goal and not d.eliminated])
    success_percentage = (num_successful / total_initial_drones) * 100 if
total_initial_drones > 0 else 0
    average_time = 0.0
    if num_successful > 0:
        times = [d.time_to_goal for d in successful_drones if d.time_to_goal >=
 0]
        if times:
            average_time = np.mean(times)
```

```
        print(f"Simulation␣Run␣Finished:␣Successful␣Drones:␣{num_successful}/{
    total_initial_drones}␣({success_percentage:.2f}%),␣"
              f"Eliminated␣(approx):␣{num_eliminated},␣Average␣Time␣(Successful):␣{
    average_time:.2f}␣s")
        return num_successful, num_eliminated, success_percentage, average_time


class SimulatorGUI:
    """Manages the Tkinter GUI for the drone swarm simulator."""
    def __init__(self, master):
        self.master = master
        master.title("Drone␣Swarm␣Simulator")
        self.swarm = Swarm(num_drones=20)
        self.swarm.gui = self
        self.running = False
        self.simulation_thread = None

        self.control_frame = ttk.Frame(master)
        self.control_frame.pack(side=tk.LEFT, padx=10, pady=10, fill=tk.Y)

        ttk.Label(self.control_frame, text="Simulation␣Mode:").pack(pady=5)
        self.mode_var = tk.StringVar(value=self.swarm.mode)
        self.pattern_mode_radio = ttk.Radiobutton(self.control_frame, text="Pattern
    ␣Formation",
                                                  variable=self.mode_var, value='
    pattern_formation',
                                                  command=self.change_mode)
        self.pattern_mode_radio.pack(pady=2)
        self.avoidance_mode_radio = ttk.Radiobutton(self.control_frame, text="
    Obstacle␣Avoidance",
                                                    variable=self.mode_var, value='
    obstacle_avoidance',
                                                    command=self.change_mode)
        self.avoidance_mode_radio.pack(pady=2)

        ttk.Label(self.control_frame, text="Number␣of␣Drones:").pack(pady=5)
        self.num_drones_var = tk.IntVar(value=self.swarm.num_drones)
        self.num_drones_entry = ttk.Entry(self.control_frame, textvariable=self.
    num_drones_var)
        self.num_drones_entry.pack(pady=5)
        self.num_drones_entry.bind("<Return>", self.update_num_drones)
        self.pattern_frame = ttk.Frame(self.control_frame)
        self.pattern_frame.pack(pady=10)
        ttk.Label(self.pattern_frame, text="Select␣Pattern:").pack(pady=5)
        self.pattern_var = tk.StringVar()
        self.pattern_dropdown = ttk.Combobox(self.pattern_frame, textvariable=self.
    pattern_var,
                                             values=["circle", "star", "line", "
    square", "triangle", "1", "S"])
        self.pattern_dropdown.pack(pady=5)
        self.pattern_dropdown.set("circle")
        self.pattern_dropdown.bind("<<ComboboxSelected>>", self.change_pattern)
        self.avoidance_settings_frame = ttk.Frame(self.control_frame)
        ttk.Label(self.avoidance_settings_frame, text="PID␣Gains:").pack(pady=5)
        ttk.Label(self.avoidance_settings_frame, text="Kp:").pack(pady=2)
        self.kp_var = tk.DoubleVar(value=self.swarm.drones[0].Kp_pid if self.swarm.
    drones else 0.8)
        self.kp_entry = ttk.Entry(self.avoidance_settings_frame, textvariable=self.
    kp_var)
        self.kp_entry.pack(pady=2)
        self.kp_entry.bind("<Return>", self.update_pid_gains)
        ttk.Label(self.avoidance_settings_frame, text="Ki:").pack(pady=2)
```

```
        self.ki_var = tk.DoubleVar(value=self.swarm.drones[0].Ki_pid if self.swarm.
drones else 0.01)
        self.ki_entry = ttk.Entry(self.avoidance_settings_frame, textvariable=self.
ki_var)
        self.ki_entry.pack(pady=2)
        self.ki_entry.bind("<Return>", self.update_pid_gains)
        ttk.Label(self.avoidance_settings_frame, text="Kd:").pack(pady=2)
        self.kd_var = tk.DoubleVar(value=self.swarm.drones[0].Kd_pid if self.swarm.
drones else 0.5)
        self.kd_entry = ttk.Entry(self.avoidance_settings_frame, textvariable=self.
kd_var)
        self.kd_entry.pack(pady=2)
        self.kd_entry.bind("<Return>", self.update_pid_gains)
        ttk.Label(self.avoidance_settings_frame, text="Simulation␣Duration␣(s):").
pack(pady=5)
        self.sim_duration_var = tk.DoubleVar(value=self.swarm.simulation_duration)
        self.sim_duration_entry = ttk.Entry(self.avoidance_settings_frame,
textvariable=self.sim_duration_var)
        self.sim_duration_entry.pack(pady=5)
        self.sim_duration_entry.bind("<Return>", self.update_sim_duration)
        ttk.Label(self.control_frame, text="Simulation␣Speed:").pack(pady=5)
        self.speed_var = tk.DoubleVar(value=1.0)
        self.speed_slider = ttk.Scale(self.control_frame, from_=0.1, to=5.0,
                                      orient=tk.HORIZONTAL, variable=self.speed_var
)
        self.speed_slider.pack(pady=5)
        self.speed_slider.bind("<Motion>", self.update_speed)

        self.start_button = ttk.Button(self.control_frame, text="Start␣Simulation",
 command=self.start_simulation)
        self.start_button.pack(pady=10)

        self.stop_button = ttk.Button(self.control_frame, text="Stop␣Simulation",
                                      command=self.stop_simulation, state=tk.
DISABLED)
        self.stop_button.pack(pady=5)
        self.info_label = ttk.Label(self.control_frame, text="")
        self.info_label.pack(pady=10)
        self.figure_swarm = plt.Figure(figsize=(8, 6), dpi=100)
        self.ax_swarm = self.figure_swarm.add_subplot(111, projection='3d')
        self.canvas_swarm = FigureCanvasTkAgg(self.figure_swarm, master=master)
        self.canvas_widget_swarm = self.canvas_swarm.get_tk_widget()
        self.canvas_widget_swarm.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True,
padx=10)
        self.update_ui_based_on_mode()
        self.plot_swarm()
        if self.swarm.drones:
            self.kp_var.set(self.swarm.drones[0].Kp_pid)
            self.ki_var.set(self.swarm.drones[0].Ki_pid)
            self.kd_var.set(self.swarm.drones[0].Kd_pid)

    def change_mode(self):
        """Handles the change in simulation mode selection."""
        selected_mode = self.mode_var.get()
        self.swarm.set_mode(selected_mode)
        self.update_ui_based_on_mode()
        self.plot_swarm()

    def update_ui_based_on_mode(self):
        """Updates the visibility of UI elements based on the current simulation
mode."""
        if self.swarm.mode == 'pattern_formation':
            self.pattern_frame.pack(pady=10)
```

```
            self.avoidance_settings_frame.pack_forget()
            self.info_label.config(text="Pattern␣Formation␣Mode")
        else:
            self.pattern_frame.pack_forget()
            self.avoidance_settings_frame.pack(pady=10)
            self.info_label.config(text="Obstacle␣Avoidance␣Mode\nReady␣to␣start.")

        self.canvas_widget_swarm.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True,
    padx=10)

    def update_num_drones(self, event=None):
        """Updates the number of drones in the swarm."""
        try:
            new_num = self.num_drones_var.get()
            if new_num >= 0:
                self.swarm.resize_swarm(new_num)
                if self.swarm.drones:
                    self.kp_var.set(self.swarm.drones[0].Kp_pid)
                    self.ki_var.set(self.swarm.drones[0].Ki_pid)
                    self.kd_var.set(self.swarm.drones[0].Kd_pid)
                else:
                    self.kp_var.set(0.8)
                    self.ki_var.set(0.01)
                    self.kd_var.set(0.5)
                self.plot_swarm()

        except ValueError:
            print("Invalid␣number␣of␣drones")

    def update_sim_duration(self, event=None):
        """Updates the duration for obstacle avoidance simulations."""
        try:
            new_duration = self.sim_duration_var.get()
            if new_duration > 0:
                self.swarm.simulation_duration = new_duration
        except ValueError:
            print("Invalid␣simulation␣duration")

    def change_pattern(self, event=None):
        """Changes the target pattern for the drones."""
        selected_pattern = self.pattern_var.get()
        self.swarm.set_pattern(selected_pattern)

    def update_speed(self, event=None):
        """Updates the simulation speed."""
        self.swarm.simulation_speed = self.speed_slider.get()

    def plot_swarm(self):
        """Plots the current state of the swarm and obstacles."""
        self.ax_swarm.cla()

        active_drones = [d for d in self.swarm.drones if not d.eliminated and not d
    .reached_goal]
        if active_drones:
            x = [drone.position[0] for drone in active_drones]
            y = [drone.position[1] for drone in active_drones]
            z = [drone.position[2] for drone in active_drones]
            self.ax_swarm.scatter(x, y, z, marker='o', c='b', label=f'Drones␣({len(
    active_drones)}␣active)')

        reached_drones = [d for d in self.swarm.drones if d.reached_goal]
        if reached_drones:
            x_r = [drone.position[0] for drone in reached_drones]
```

```
            y_r = [drone.position[1] for drone in reached_drones]
            z_r = [drone.position[2] for drone in reached_drones]
            self.ax_swarm.scatter(x_r, y_r, z_r, marker='*', c='g', s=100, label=f'
Reached␣Goal␣({len(reached_drones)})')

        eliminated_drones = [d for d in self.swarm.drones if d.eliminated]

        if self.swarm.mode == 'obstacle_avoidance':
            for pillar in self.swarm.obstacles:
                theta = np.linspace(0, 2 * np.pi, 10)
                z_cyl = np.linspace(pillar.base_position[2], pillar.base_position
[2] + pillar.height, 2)
                Z_mesh, THETA_mesh = np.meshgrid(z_cyl, theta)
                X_cyl = pillar.base_position[0] + pillar.radius * np.cos(THETA_mesh
)
                Y_cyl = pillar.base_position[1] + pillar.radius * np.sin(THETA_mesh
)
                self.ax_swarm.plot_surface(X_cyl, Y_cyl, Z_mesh, color='gray',
alpha=0.6, rstride=1, cstride=1)

            start_size = self.swarm.start_box_size
            sb_center = self.swarm.start_box_center
            x_s = [sb_center[0]-s/2 for s in [start_size, -start_size, -start_size,
 start_size, start_size]]
            y_s = [sb_center[1]-s/2 for s in [start_size, start_size, -start_size,
-start_size, start_size]]
            z_s_bot = [sb_center[2]-start_size/2] * 5
            z_s_top = [sb_center[2]+start_size/2] * 5
            self.ax_swarm.plot(x_s, y_s, z_s_bot, color='green', linestyle='--',
label='Start␣Box')
            self.ax_swarm.plot(x_s, y_s, z_s_top, color='green', linestyle='--')
            for i in range(4):
                self.ax_swarm.plot([x_s[i], x_s[i]], [y_s[i],y_s[i]], [z_s_bot[i],
z_s_top[i]], color='green', linestyle='--')

            end_size = self.swarm.end_box_size
            eb_center = self.swarm.end_box_center
            x_e = [eb_center[0]-s/2 for s in [end_size, -end_size, -end_size,
end_size, end_size]]
            y_e = [eb_center[1]-s/2 for s in [end_size, end_size, -end_size, -
end_size, end_size]]
            z_e_bot = [eb_center[2]-end_size/2] * 5
            z_e_top = [eb_center[2]+end_size/2] * 5
            self.ax_swarm.plot(x_e, y_e, z_e_bot, color='red', linestyle='--',
label='End␣Box')
            self.ax_swarm.plot(x_e, y_e, z_e_top, color='red', linestyle='--')
            for i in range(4):
                self.ax_swarm.plot([x_e[i], x_e[i]], [y_e[i],y_e[i]], [z_e_bot[i],
z_e_top[i]], color='red', linestyle='--')


        plot_range = self.swarm.area_size / 2
        self.ax_swarm.set_xlim([-plot_range, plot_range])
        self.ax_swarm.set_ylim([-plot_range, plot_range])
        max_z_limit = max(self.swarm.area_size / 4 + 15/2 + 5, self.swarm.area_size
 / 2 + 5)
        self.ax_swarm.set_zlim([0, max_z_limit])
        self.ax_swarm.set_xlabel('X')
        self.ax_swarm.set_ylabel('Y')
        self.ax_swarm.set_zlabel('Z')
        self.ax_swarm.set_title(f"Drone␣Swarm␣Simulation␣({self.swarm.mode.replace
('_',␣'␣').title()}␣Mode)")
        if self.ax_swarm.get_legend_handles_labels()[1]:
```

```
            self.ax_swarm.legend()
        self.canvas_swarm.draw()

    def simulation_loop(self):
        """The main loop for running the simulation steps and updating the plot."""
        base_dt = 0.05

        while self.running:
            start_time_loop = time.time()
            dt = base_dt * self.swarm.simulation_speed
            self.swarm.update_swarm(dt)
            if self.swarm.drones or any(d.reached_goal for d in self.swarm.drones):
                self.master.after(1, self.plot_swarm)

            end_time_loop = time.time()
            elapsed_frame_time = end_time_loop - start_time_loop
            sleep_time = base_dt - elapsed_frame_time
            if sleep_time > 0:
                time.sleep(sleep_time / self.swarm.simulation_speed)
            elif sleep_time < 0:
                time.sleep(0.001)


    def start_simulation(self):
        """Starts or restarts the simulation."""
        if not self.running:
            self.running = True
            self.start_button.config(state=tk.DISABLED)
            self.stop_button.config(state=tk.NORMAL)

            self.swarm.reset_swarm()
            if self.swarm.drones:
                self.kp_var.set(self.swarm.drones[0].Kp_pid)
                self.ki_var.set(self.swarm.drones[0].Ki_pid)
                self.kd_var.set(self.swarm.drones[0].Kd_pid)
            self.plot_swarm()
            if self.simulation_thread is None or not self.simulation_thread.
    is_alive():
                self.simulation_thread = threading.Thread(target=self.
    simulation_loop)
                self.simulation_thread.daemon = True
                self.simulation_thread.start()

            if self.swarm.mode == 'obstacle_avoidance':
                self.info_label.config(text=f"Obstacle␣Avoidance␣Mode\nRunning...")
            else:
                self.info_label.config(text=f"Pattern␣Formation␣Mode\nRunning...")

    def end_simulation_run(self):
        """Called by Swarm when an obstacle avoidance simulation run finishes."""
        if self.swarm.mode == 'obstacle_avoidance':
            num_successful, num_eliminated, success_percentage, average_time = self
    .swarm.calculate_run_results()

            self.info_label.config(text=f"Obstacle␣Avoidance␣Mode\nRun␣Finished:\n"
                                        f"Successful:␣{num_successful}/{self.swarm.
    num_drones}␣({success_percentage:.2f}%)\n"
                                        f"Eliminated␣(approx):␣{num_eliminated}\n"
                                        f"Avg␣Time␣(Successful):␣{average_time:.2f}
    ␣s")
        self.running = False
        self.start_button.config(state=tk.NORMAL)
        self.stop_button.config(state=tk.DISABLED)
```

```
        print("Simulation␣run␣finished␣(or␣ended).")


    def stop_simulation(self):
        """Stops the currently running simulation."""
        if self.running:
            self.running = False
            self.start_button.config(state=tk.NORMAL)
            self.stop_button.config(state=tk.DISABLED)
            print("Simulation␣stopped␣manually.")
            if self.swarm.mode == 'obstacle_avoidance':
                self.info_label.config(text="Obstacle␣Avoidance␣Mode\nSimulation␣
    stopped.")
            else:
                self.info_label.config(text="Pattern␣Formation␣Mode\nSimulation␣
    stopped.")

    def update_pid_gains(self, event=None):
        """Updates PID gains for all drones."""
        try:
            new_kp = self.kp_var.get()
            new_ki = self.ki_var.get()
            new_kd = self.kd_var.get()
            if new_kp >= 0 and new_ki >= 0 and new_kd >= 0:
                for drone in self.swarm.drones:
                    drone.Kp_pid = new_kp
                    drone.Ki_pid = new_ki
                    drone.Kd_pid = new_kd
                print(f"Updated␣PID␣gains:␣Kp={new_kp},␣Ki={new_ki},␣Kd={new_kd}")
        except ValueError:
            print("Invalid␣PID␣gain␣value")


if __name__ == "__main__":
    root = tk.Tk()
    gui = SimulatorGUI(root)
    def on_closing():
        if gui.running:
            gui.stop_simulation()
        if gui.simulation_thread and gui.simulation_thread.is_alive():
            try:
                gui.simulation_thread.join(timeout=0.1)
            except Exception as e:
                print(f"Error␣joining␣simulation␣thread:␣{e}")
        root.quit()

    root.protocol("WM_DELETE_WINDOW", on_closing)
    root.mainloop()
```